Product Delivery Prediction Report

Hayden Sterling

2025-06-03

This project, completed during a statistics course at Northwestern, aims to classify the binary outcome of product deliveries as either on time and complete (1), or not on time and complete (0). The dataset was provided to me by the course instructors - it contains 28 features and over 20,000 instances of product deliveries from an unknown company. The features contain a wide range of information, spanning categorical, numerical, and date-time datatypes. The metric I attempt to maximize is accuracy. Over the course of two academic quarters, I cleaned and imputed the data, conducted feature engineering, and attempted a variety of classical ML models for prediction, including Logistic Regression, Random Forest, K-Nearest Neighbors (KNN), and Extreme Gradient Boosted Decision Trees (XGBoost). For the first academic quarter, I focused solely on building a robust and interpretable Logistic Regression model using regularization and hyperparameter tuning to balance bias and variance. During the second academic quarter, I experimented with a variety of classical ML approaches, including stacking. This report includes work from both academic quarters. Ultimately, I conclude that a tuned, regularized ensemble stacking model combining all of the previously listed models while using a Logistic Regression meta-model had the highest performance, with a test accuracy of 87.27%, using a 90-10 train-test split. However, the XGBoost model obtains a comparable accuracy, while remaining computationally more efficient than the stacking model.

0.1 Problem Statement (and Recognition of Evaluation Metric Trade-Off)

The goal of this project is to predict the outcomes of product deliveries from an unknown company, classifying them as either on-time and complete or not on-time and complete. The dataset includes >20,000 rows and 28 features, with the response variable being `ON_TIME_AND_COMPLETE`, where a 1 corresponds to an on-time and completed delivery. I will aim to maximize accuracy, as that is what was outlined in the project assignment; however, it is important to know that this might not be the best approach from an industry standpoint. Accuracy ignores the complexities surrounding the different types of errors in binary classification.

If this project were completed at a company looking to predict the outcome of future deliveries, the model committing a false positive means telling a customer that the delivery will occur on-time when it actually won't, and the model committing a false negative means sending out a warning to the customer that the delivery could be late or incomplete, when it actually will occur

on-time. Depending on the business's needs and their promises to the customers, it may be better to optimize the model for a different metric, such as precision (when trying to minimize false positives), recall (when trying to minimize false negatives), or F1-score (when trying to strike a balance between the two).

Since this is a binary classification problem, I will take a variety of approaches that include Logistic Regression, KNN, decision trees, and ensemble methods (such as random forests or gradient boosting). I will optimize each of these models individually, experimenting with various hyperparameter tuning methods such as grid search, random search, Bayesian search, and Optuna. Finally, I will build a stacking model that combines the previously listed base learners using a meta-learner. I hope to find the best-performing model in terms of accuracy among these.

```
import pandas as pd
import numpy as np

from sklearn.model_selection import cross_val_score,train_test_split, GridSearchCV, Para
StratifiedKFold, RandomizedSearchCV, cross_val_predict
from sklearn.metrics import root_mean_squared_error, mean_squared_error,r2_score,roc_cur
from sklearn.model_selection import KFold
from sklearn.tree import DecisionTreeRegressor,DecisionTreeClassifier
from sklearn.ensemble import VotingRegressor, VotingClassifier, StackingRegressor, \
StackingClassifier, GradientBoostingRegressor,GradientBoostingClassifier, BaggingRegress
BaggingClassifier,RandomForestRegressor,RandomForestClassifier,AdaBoostRegressor,AdaBoos
from sklearn.linear_model import LinearRegression,LogisticRegression, Ridge, ElasticNetC
from sklearn.neighbors import KNeighborsRegressor, KNeighborsClassifier
from sklearn.impute import SimpleImputer
from xgboost import XGBRegressor, XGBClassifier
from lightgbm import LGBMClassifier, LGBMRegressor
from catboost import CatBoostClassifier, CatBoostRegressor
from sklearn.preprocessing import StandardScaler, OneHotEncoder, PolynomialFeatures
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from skopt.space import Real, Categorical, Integer
from skopt import BayesSearchCV
import optuna
import optuna.visualization as vis

import seaborn as sns
import matplotlib.pyplot as plt
import itertools as it
import time as time
%matplotlib inline
import warnings
warnings.filterwarnings("ignore", category=FutureWarning)

import plotly.io as pio
```

```
pio.renderers.default = "plotly_mimetype"
```

```
# Train X and y

train_X = pd.read_csv('../data/train_X.csv')
train_y = pd.read_csv('../data/train_y.csv')
train = train_X.merge(train_y, on='ID')
```

0.2 Exploratory Data Analysis (EDA) and Feature Engineering

The data for this prediction problem has 28 features. The first step in data preprocessing was to search for and remove any unnecessary features. I was able to immediately remove 4 features using my intuition. These features included:

- `ID` is unique for every single observation, essentially serving as an indexer, which is unhelpful for predictive purposes.
- `DIVISION_NUMBER` contains information that is perfectly reflected in another feature, `DIVISION_CODE`, as they each have three unique values that are essentially identical. They create identical columns when one-hot-encoded, and thus will be dropped to avoid multicollinearity issues.
- `RESERVABLE_INDICATOR` and `PRODUCT_STATUS` both only have one unique value, rendering them useless for prediction.

Later on in my analysis, it was determined that 3 more features were unhelpful for reasons more complicated than previously stated.

- `PURCHASE_ORDER_DUE_DATE` and `ORDER_DATE` contain dates about when the order was placed and when it was due. When these columns' years, months, days, and weekdays are extracted, they are unhelpful to the model as a whole, failing to increase goodness-of-fit or accuracy.
- `PURCHASE_ORDER_TYPE` was deemed not useful when conducting data visualization, as it appeared to have almost zero relationship with the log-odds of the response variable.

```
### --------------------- DROPPING COLUMNS THAT AREN'T USEFUL ---------------------

cols_to_drop = [
    'ID', # identification number unique to each delivery (not useful)
    'PURCHASE_ORDER_DUE_DATE', # found the datetime columns to be not useful
    'ORDER_DATE', # found the datetime columns to be not useful
    'DIVISION_NUMBER', # information is perfectly reflected in DIVISION_CODE (identical
    'RESERVABLE_INDICATOR', # only have one unique value
    'PRODUCT_STATUS', # only has one unique value
    'PURCHASE_ORDER_TYPE', # deemed not useful in data visualization later on
]

train.drop(columns=cols_to_drop, inplace=True)
```

In addition to discarding the above features, other features that appeared to be numeric actually served the model better as categorical. These include:

- `COMPANY_VENDOR_NUMBER` is a number/code representing the vendor that provided the food. Since this would likely have no linear relationship with log-odds, it was made categorical. Although this column has a high number of unique values, which could harm the data's dimensionality, it greatly increased the accuracy of the model when turned categorical, so it was kept.

- `ORDER_DAY_OF_WEEK` and `DUE_DATE_WEEKDAY` are numerical columns ranging from 1-7, representing the day of the week for the delivery. Again, it was found that this had no linear relationship with log-odds, but performed much better when turned categorical.
- `GIVEN_TIME_TO_LEAD_TIME_RATIO` seems like it should remain numerical since it is a ratio. However, after performing data visualization, it was found that it did not hold a linear, quadratic, etc. relationship with log-odds, but still contained information relevant to the prediction. Since it had a relatively low number of unique values, turning this column categorical did not severely damage the dimensionality of the data, and increased prediction accuracy.

```
### --------------------- CHANGING DTYPE OF COLUMNS ---------------------

cat_preds = [
    'DIVISION_CODE', # already categorical
    'COMPANY_VENDOR_NUMBER', # represents a vendor number, works better categorically
    'ORDER_DAY_OF_WEEK', # day of the week, works better categorically
    'DUE_DATE_WEEKDAY', # day of the week, works better categorically
    'GIVEN_TIME_TO_LEAD_TIME_RATIO',
]
train[cat_preds] = train[cat_preds].astype('str')

# defining the set of numerical columns
num_preds = [col for col in train.columns if col not in cat_preds + ["ON_TIME_AND_COMPLE
```

After removing certain columns in the code cell above, I imputed the missing data in the dataset using a linear regression. For each column with missing values, I found the top 5 most highly correlated other columns and used them as predictors in a linear regression to predict the values of the column with missingness issues.

```
#### -------------------- MISSING DATA IMPUTATION --------------------

# Detecting the columns with missing values
columns_missingness = train.isna().sum()[train.isna().sum() != 0].index
corr_matrix = train[num_preds].corr().abs()

# For the columns with missing values, finding which columns it has the best correlation
for col in columns_missingness:
```

```
    # Getting the five variables with the highest correlation as X in the LR
    vars_with_highest_corr = list(corr_matrix[col].drop(columns_missingness).sort_values

    # Building a linear regression to impute col using var_with_highest_corr
    x = train[vars_with_highest_corr]
    y = train[col]
    idx_non_missing = np.isfinite(x).all(axis=1) & np.isfinite(y)
    model = LinearRegression()
    model.fit(x[idx_non_missing], y[idx_non_missing])

    # Imputing the train data
    missing_idx_train = train[col].isna()
    train.loc[missing_idx_train, col] = model.predict(train.loc[missing_idx_train, vars_
```

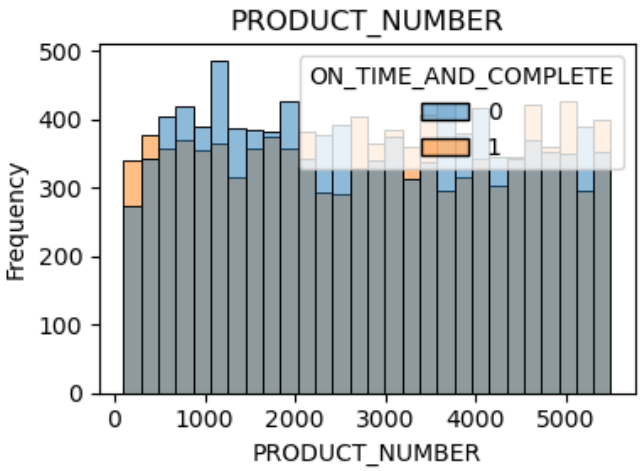
To effectively construct useful features, I visualized each numerical column in two ways:

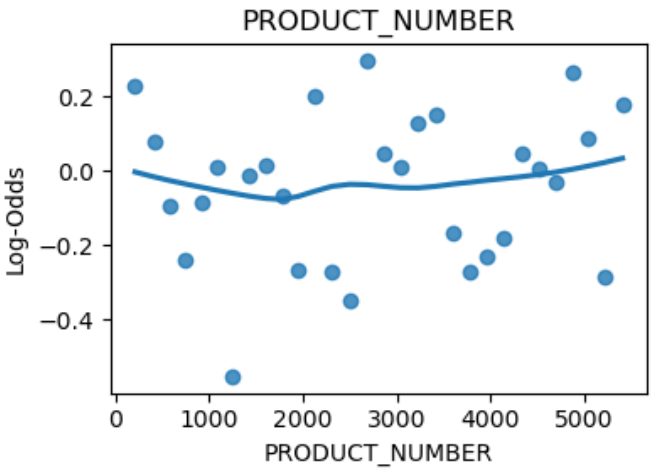
1. A histogram that displayed the distribution of the data. Each histogram contained an overlay representing the value of the response variable. This plot helped immensely in detecting which columns needed transformations, and which ones had certain values with a high percentage of the response variable being either a 0 or 1.
2. A log odds plot. By binning and grouping each column, I was able to take the average value of `ON_TIME_AND_COMPLETE`, the response variable, for each group. This allowed for effective estimation of the relationship between certain predictors and the log-odds. These plots were **crucial** in constructing strong features with high predictive power, as they made clear which variables needed certain transformations, and also which variables needed to be discarded/binned later on.

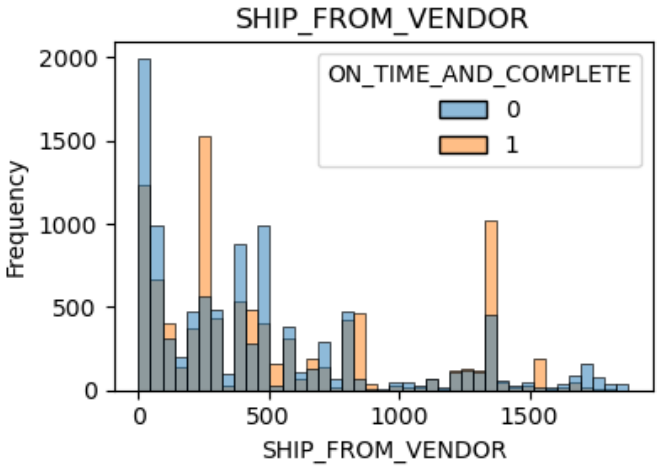
Below is the code and its resulting visualizations.

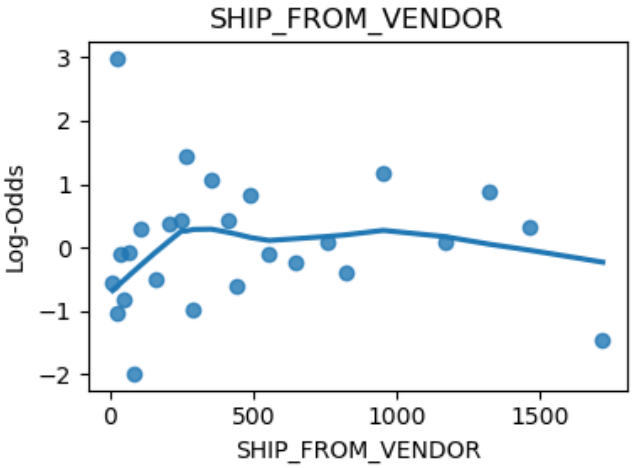
```
### ----------------- DATA VISUALIZATION -----------------

train_viz = train.copy()

# function to estimate log-odds of a group
def compute_log_odds(df, predictor, response, bins=30):

    """
    Computes an estimate of the log odds relationship for a particular column
    Args:
        - `df`: a dataframe
        - `col`: specific column to estimate the relationship of the log odds
        - `response`: the response column, ie the column that needs its log-odds estimat
    Returns: a grouped dataframe containing the estimates and bins
    """

    df['bin'] = pd.qcut(df[predictor], q=bins, duplicates='drop') # Binning the data to
    grouped = df.groupby('bin', observed=False)[response].agg(['mean'])
```

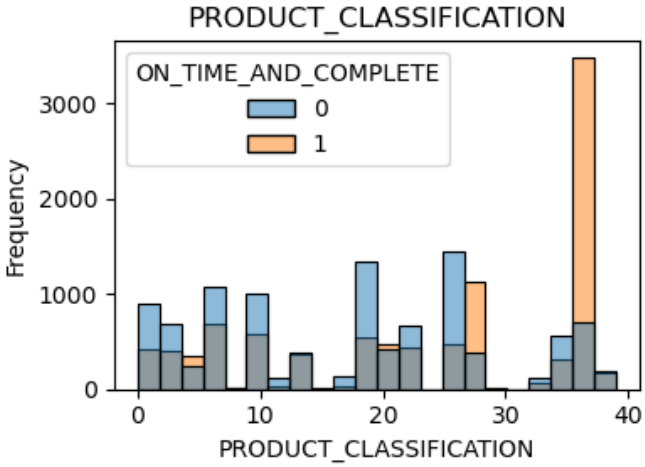
```
    # Compute log odds
    grouped['log_odds'] = np.log(grouped['mean'] / (1 - grouped['mean']))

    return grouped

# visualizing the histogram and log-odds plot of each
for col in num_preds:
    fig, axes = plt.subplots(1, 2, figsize=(8, 3))

    # Histogram of predictor on train data
    sns.histplot(data = train_viz, x=col, ax=axes[0], hue='ON_TIME_AND_COMPLETE')
    axes[0].set_title(f'{col}')
    axes[0].set_xlabel(col)
    axes[0].set_ylabel('Frequency')

    # Computing log odds for the column
    log_odds_data = compute_log_odds(train_viz, col, 'ON_TIME_AND_COMPLETE')

    # Scatterplot with trendline (regplot) of predictor on train predictor
    sns.regplot(x = log_odds_data.index.categories.mid, y = log_odds_data['log_odds'], a
    axes[1].set_title(f'{col}')
    axes[1].set_xlabel(col)
    axes[1].set_ylabel('Log-Odds')

    # Showing the plots
    plt.tight_layout()
    plt.show()
```

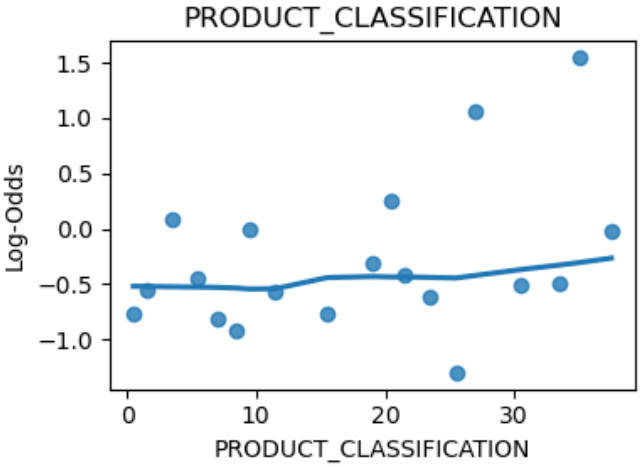


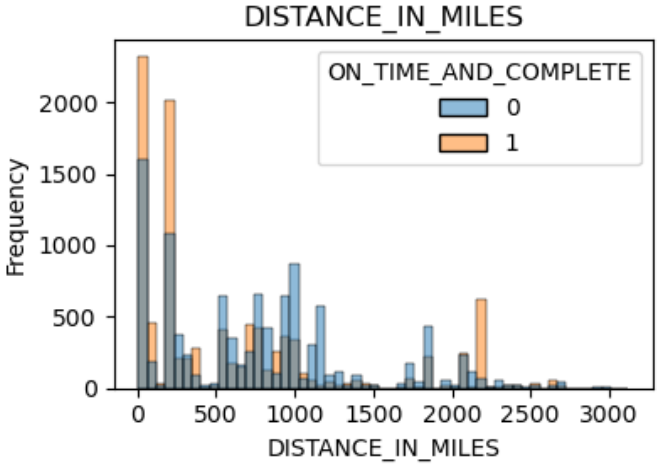


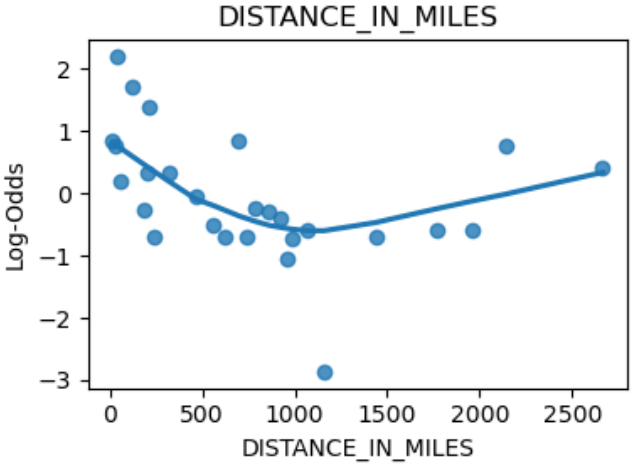

SHIP_FROM_VENDOR
ON_TIME_AND_COMPLETE
0
1
Frequency
SHIP_FROM_VENDOR

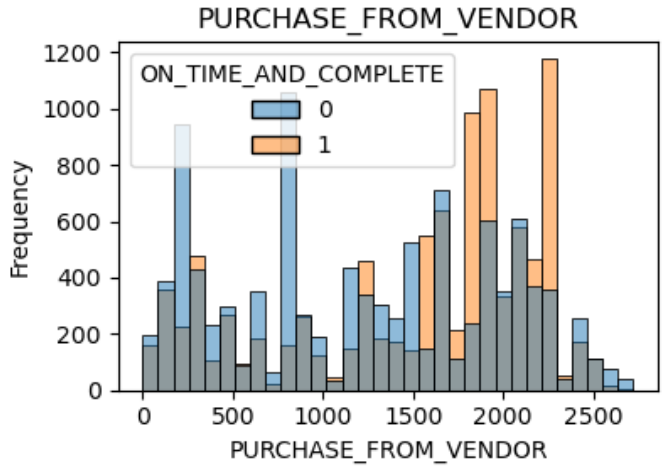

SHIP_FROM_VENDOR
Log-Odds
SHIP_FROM_VENDOR

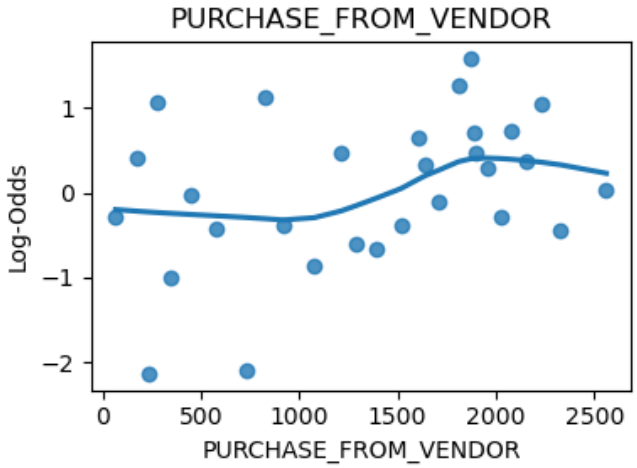

PRODUCT_CLASSIFICATION
ON_TIME_AND_COMPLETE
0
1
Frequency
PRODUCT_CLASSIFICATION

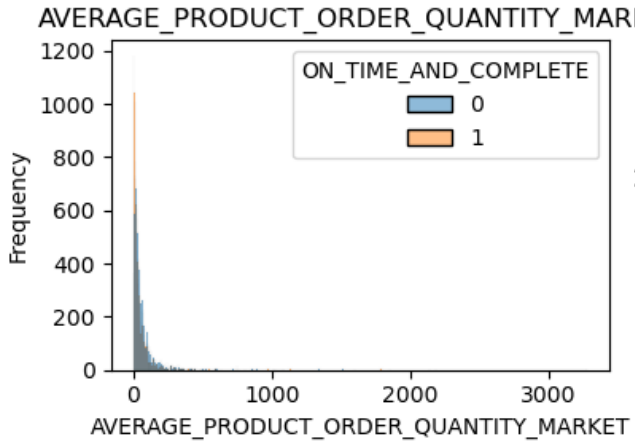

PRODUCT_CLASSIFICATION
Log-Odds
PRODUCT_CLASSIFICATION

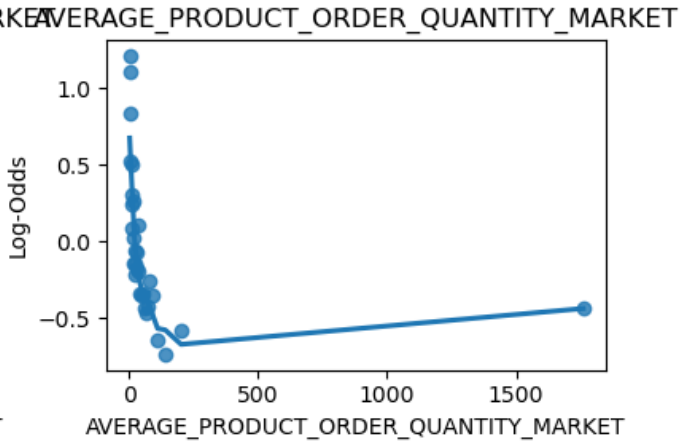

DISTANCE_IN_MILES
ON_TIME_AND_COMPLETE
0
1
Frequency
DISTANCE_IN_MILES

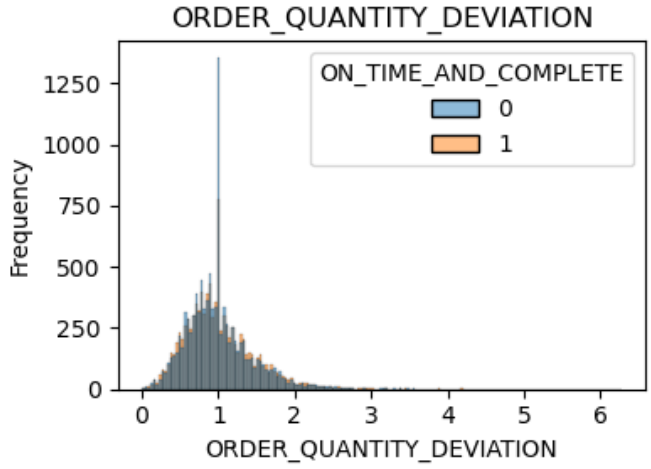

DISTANCE_IN_MILES
Log-Odds
DISTANCE_IN_MILES

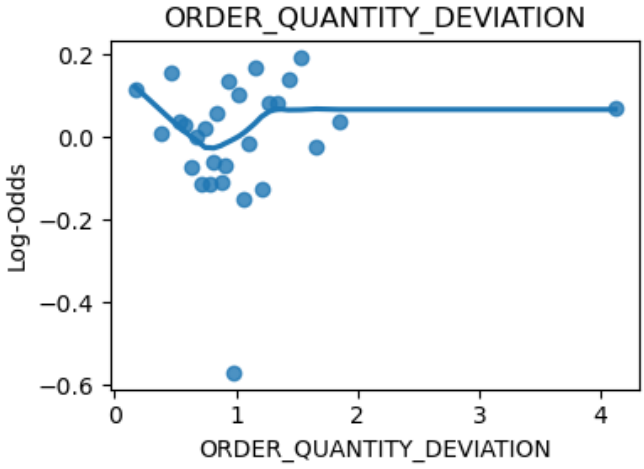

PURCHASE_FROM_VENDOR
ON_TIME_AND_COMPLETE
0
1
Frequency
PURCHASE_FROM_VENDOR

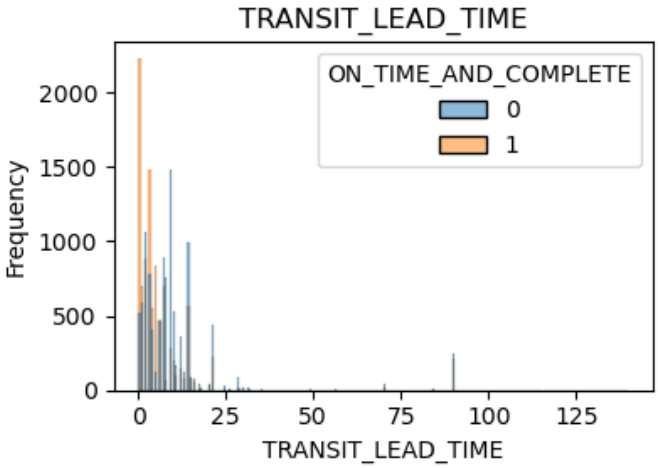

PURCHASE_FROM_VENDOR
Log-Odds
PURCHASE_FROM_VENDOR

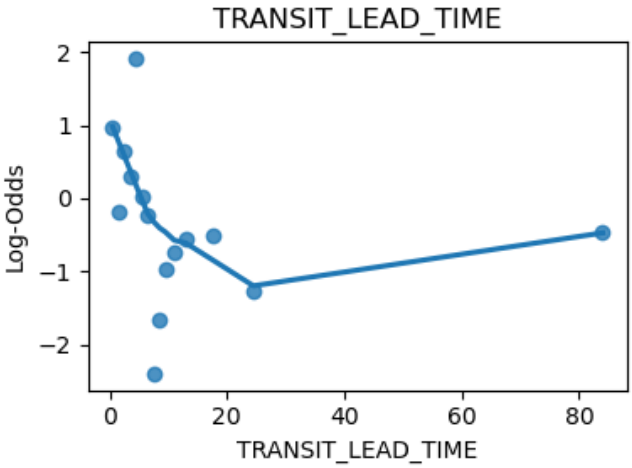

AVERAGE_PRODUCT_ORDER_QUANTITY_MARKET
ON_TIME_AND_COMPLETE
0
1
Frequency
AVERAGE_PRODUCT_ORDER_QUANTITY_MARKET

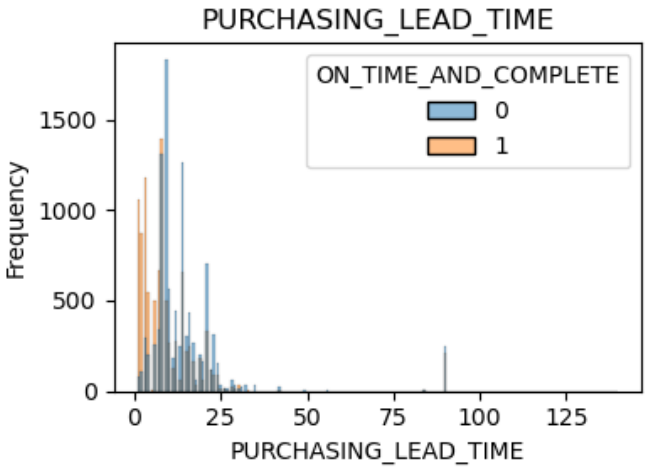

AVERAGE_PRODUCT_ORDER_QUANTITY_MARKET
Log-Odds
AVERAGE_PRODUCT_ORDER_QUANTITY_MARKET

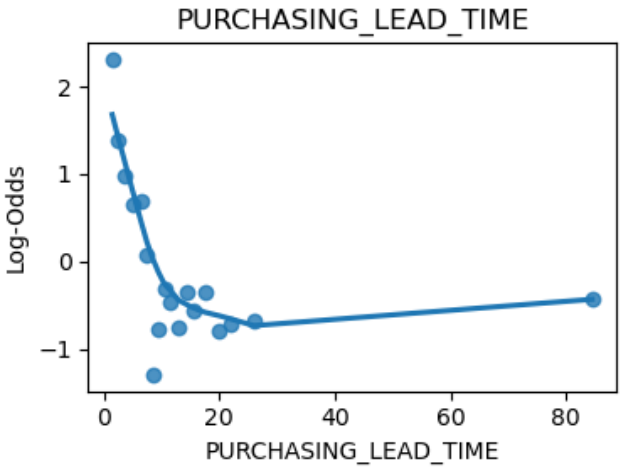

ORDER_QUANTITY_DEVIATION
ON_TIME_AND_COMPLETE
0
1
Frequency
ORDER_QUANTITY_DEVIATION

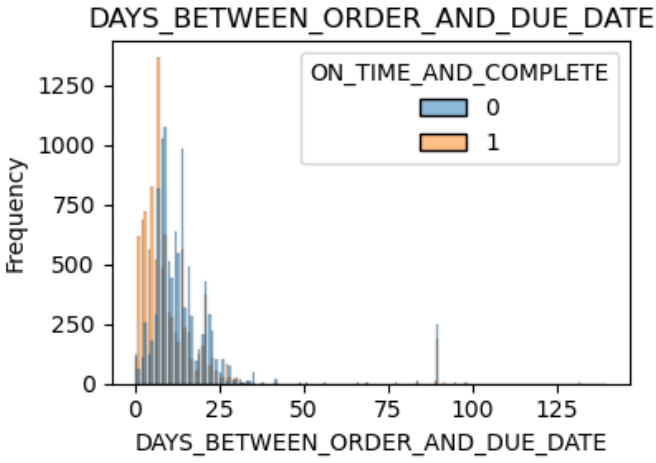

ORDER_QUANTITY_DEVIATION
Log-Odds
ORDER_QUANTITY_DEVIATION

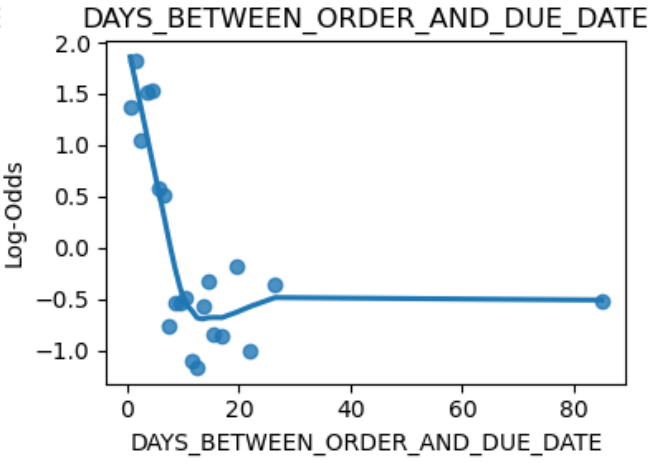

TRANSIT_LEAD_TIME
ON_TIME_AND_COMPLETE
0
1
Frequency
TRANSIT_LEAD_TIME

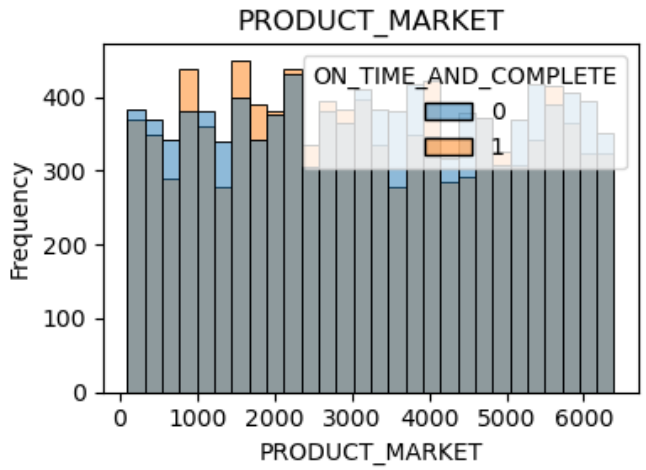

TRANSIT_LEAD_TIME
Log-Odds
TRANSIT_LEAD_TIME

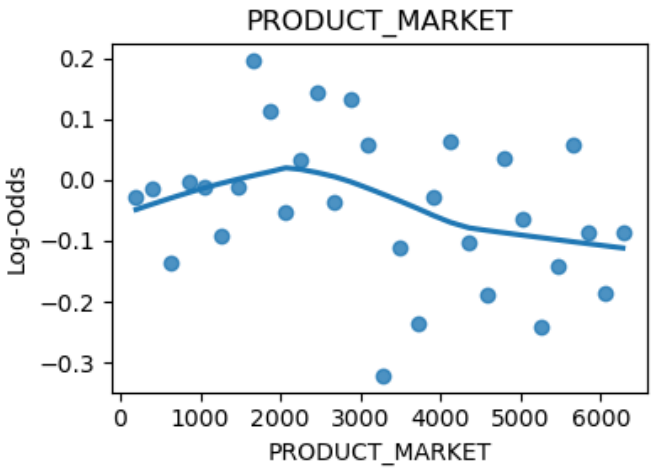

PURCHASING_LEAD_TIME
ON_TIME_AND_COMPLETE
0
1
Frequency
PURCHASING_LEAD_TIME

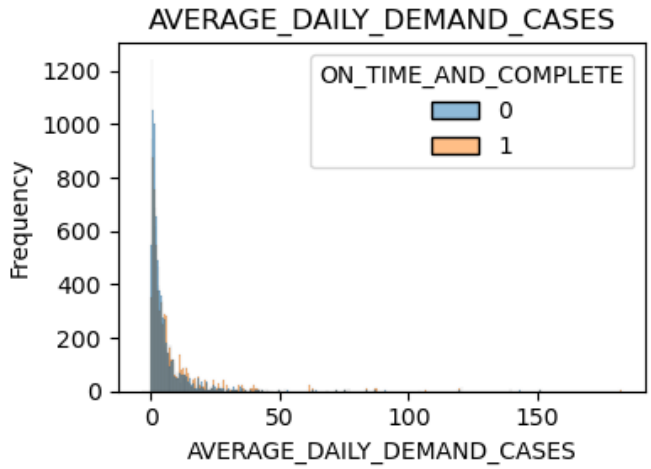

PURCHASING_LEAD_TIME
Log-Odds
PURCHASING_LEAD_TIME

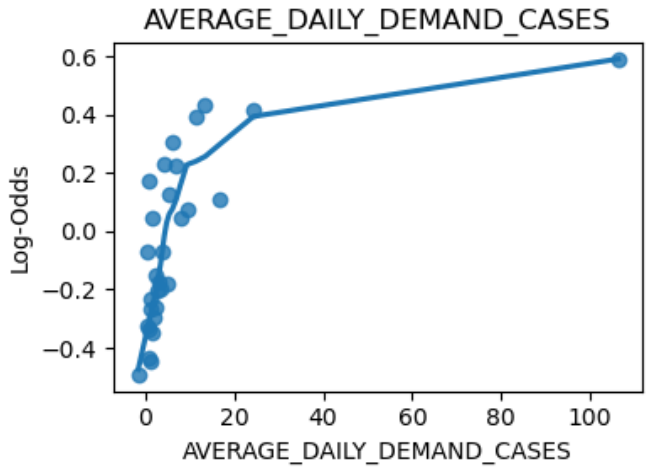

DAYS_BETWEEN_ORDER_AND_DUE_DATE
ON_TIME_AND_COMPLETE
0
1
Frequency
DAYS_BETWEEN_ORDER_AND_DUE_DATE

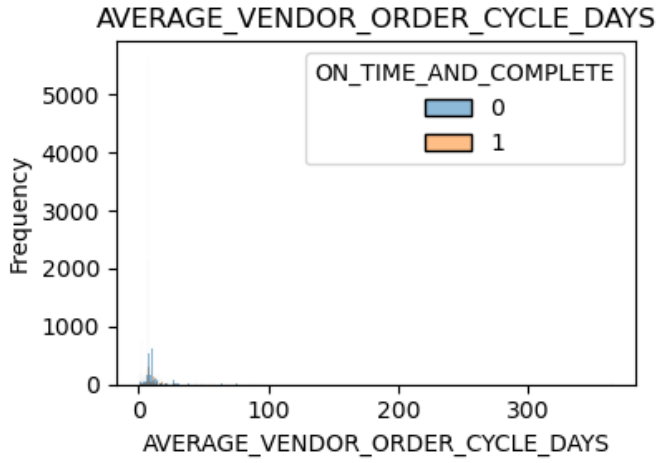

DAYS_BETWEEN_ORDER_AND_DUE_DATE
Log-Odds
DAYS_BETWEEN_ORDER_AND_DUE_DATE

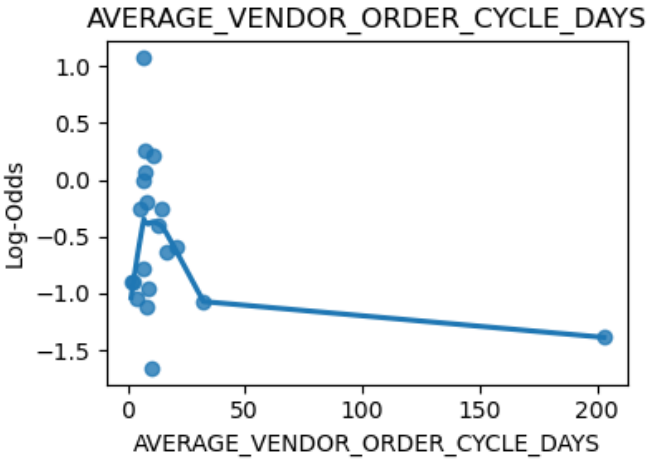

PRODUCT_MARKET
ON_TIME_AND_COMPLETE
0
1
Frequency
PRODUCT_MARKET

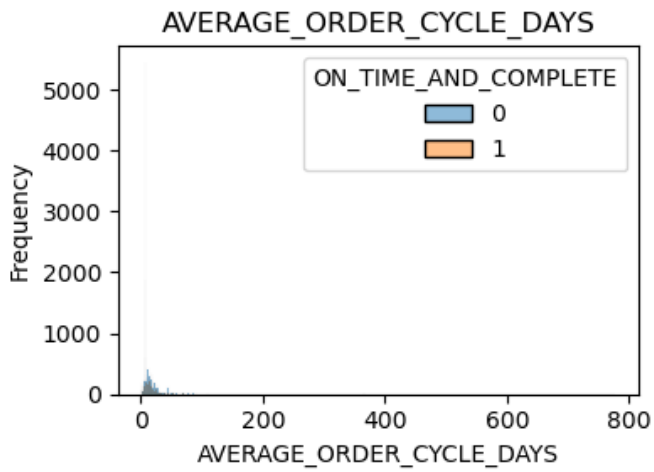

PRODUCT_MARKET
Log-Odds
PRODUCT_MARKET

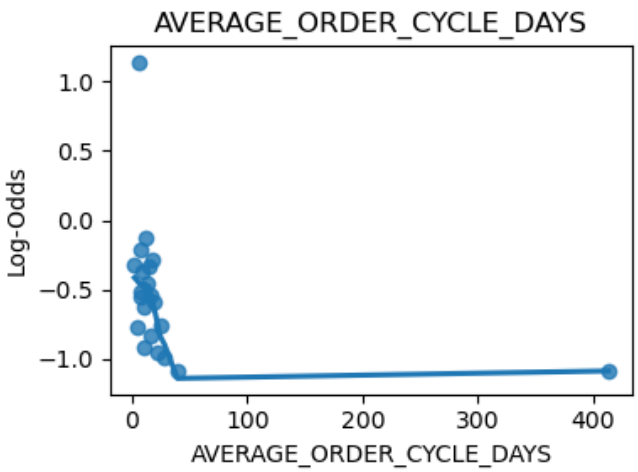

AVERAGE_DAILY_DEMAND_CASES
ON_TIME_AND_COMPLETE
0
1
Frequency
AVERAGE_DAILY_DEMAND_CASES

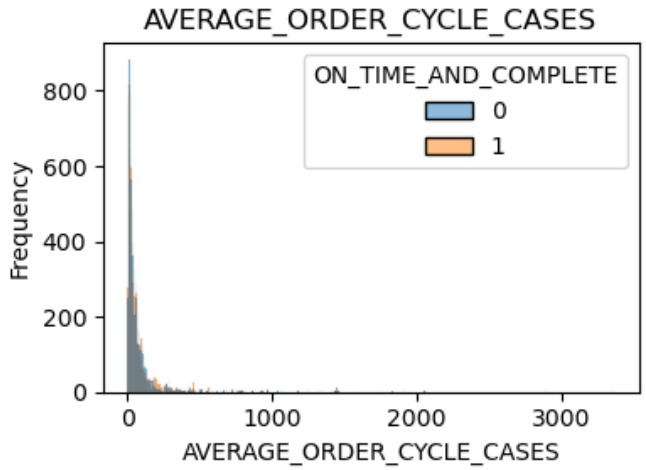

AVERAGE_DAILY_DEMAND_CASES
Log-Odds
AVERAGE_DAILY_DEMAND_CASES

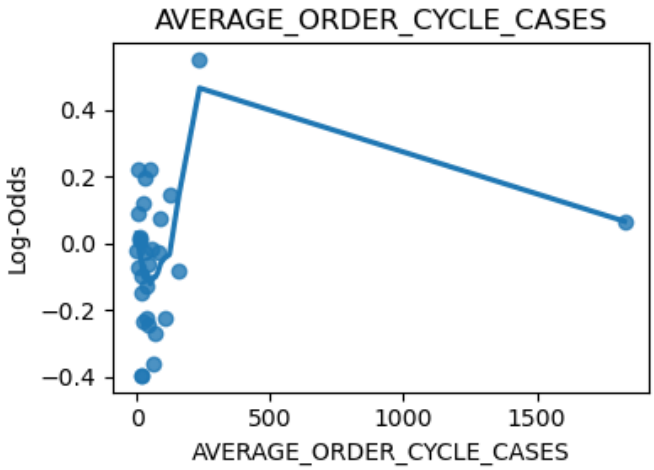

AVERAGE_VENDOR_ORDER_CYCLE_DAYS
ON_TIME_AND_COMPLETE
0
1
Frequency
AVERAGE_VENDOR_ORDER_CYCLE_DAYS

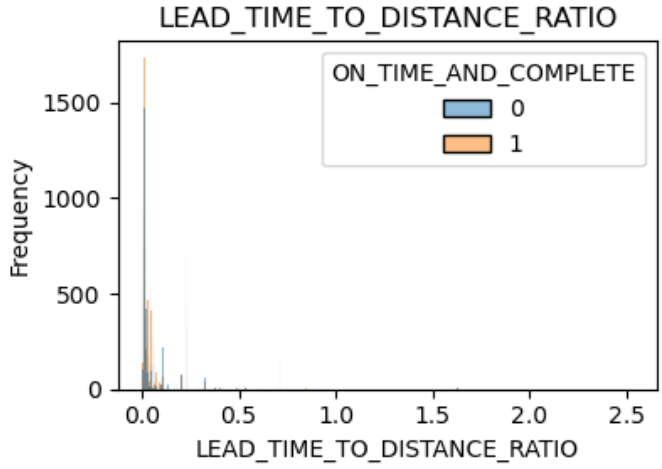

AVERAGE_VENDOR_ORDER_CYCLE_DAYS
Log-Odds
AVERAGE_VENDOR_ORDER_CYCLE_DAYS

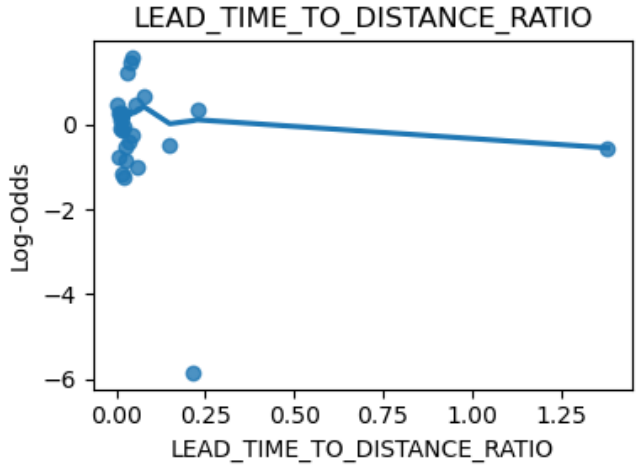












I proceeded to log-transform certain columns in the dataset. I determined which columns to transform based on the above visualizations. The columns that are log-transformed below were either heavily right-skewed, or had a scatterplot (against the log odds of the response) that reflected

a logarithmic relationship. To turn this relationship more linear, I log-transformed them. Although transforming variables is not useful for tree-based models, one of the base models for my stacking model is a logistic regression, which greatly benefits from having these transformed variables. As for the tree-based models, the transformation has little-to-no effect on them due to the way that trees are constructed (monotonic transformations do not change the outcomes of trees).

```
### ---------------- VARIABLE TRANSFORMATIONS ----------------

log_preds = [
    'DISTANCE_IN_MILES',
    'AVERAGE_PRODUCT_ORDER_QUANTITY_MARKET',
    'TRANSIT_LEAD_TIME',
    'PURCHASING_LEAD_TIME',
    'DAYS_BETWEEN_ORDER_AND_DUE_DATE',
    'AVERAGE_DAILY_DEMAND_CASES',
    'AVERAGE_VENDOR_ORDER_CYCLE_DAYS',
    'AVERAGE_ORDER_CYCLE_DAYS',
    'AVERAGE_ORDER_CYCLE_CASES',
    'LEAD_TIME_TO_DISTANCE_RATIO',
    'ORDER_QUANTITY_DEVIATION',
]

train_logged = train.copy()

# Defining a function to log transform, while accounting for negatives
def log_transformation(X):
    X = np.where(X <= -1, 0.999, X) # making sure to not pass negative values to the log
    return np.log1p(X)

# Applying the function to the specified columns
train_logged[log_preds] = train[log_preds].apply(log_transformation, axis=0)
```

In addition to log-transforming, I also binned certain columns. For similar reasons as the log-transformations, the binning of these columns was beneficial to the logistic regression base model, and had very little effect on the overall stacking model.

```
### ---------------- BINNING CERTAIN COLUMNS ----------------

def bin_column(col, method='auto', bins=None):

    # Binning according to method
    if method == 'auto':
        bins = pd.cut(train_logged[col], bins=bins, retbins=True)[1]
        train_logged[f"{col}_binned"] = pd.cut(train_logged[col], bins=bins)
    if method == 'custom':
        train_logged[f"{col}_binned"] = pd.cut(train_logged[col], bins=bins)
```

```
    # Dropping the original columns from the dataframes
    train_logged.drop(columns = [col])

    # Dropping the original column name from the num_preds list
    num_preds.remove(col)

    # Adding the new binned columns to the list of categorical variables
    global cat_preds # accessing the global variable
    cat_preds = cat_preds + [f"{col}_binned"]

    return train_logged

# Binning certain numerical columns
train_logged = bin_column(col='DAYS_BETWEEN_ORDER_AND_DUE_DATE', method='auto', bins=10)
train_logged = bin_column(col='AVERAGE_ORDER_CYCLE_DAYS', method='custom', bins=[-1, 2.0
train_logged = bin_column(col='DISTANCE_IN_MILES', method='auto', bins=20)
train_logged = bin_column(col='PURCHASE_FROM_VENDOR', method='auto', bins=10)
train_logged = bin_column(col='TRANSIT_LEAD_TIME', method='auto', bins=15)
```

The above code cells represent most of the data preprocessing and feature engineering that I completed for this prediction problem. ***The remainder of the data preprocessing was done in the model pipelines for each individual base model, which was completed slightly differently for each base model. For that reason, those steps are left to the next section.***

Below, I split the dataset into a training and testing set in order to obtain an unbiased estimate of the model's test error before each submission. I chose a test size of 0.1 to retain more training data than is typical. Since the dataset is very large, I deemed this test size appropriate, as the test set still included enough rows to provide a good estimate.

```
# Train-test split
X = train_logged.drop(columns=['ON_TIME_AND_COMPLETE'])
y = train_logged['ON_TIME_AND_COMPLETE']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.1, random_state=42
```

0.3 First Academic Quarter: Logistic Regression

This project spans two academic quarters of the STAT 303 sequence, and I spent the first quarter working on building only a Logistic Regression model. Below are my steps to building this model, which will eventually be incorporated into a stacking model along with other ML models.

0.3.1 Logistic Model Setup and Initial Training

The Logistic Regression model was built using the sklearn `Pipeline` and `ColumnTransformer` classes for effective and consistent transformations on both the training and test sets.

The pipeline for numerical columns included the creation of 2nd-order features as well as interaction terms between all features. Additionally, all numerical features were standardized using `StandardScaler` to ensure the stability of the coefficient estimates during the optimization process, which includes regularization.

The categorical pipeline simply one-hot-encodes each categorical column to prepare them for model training. Each categorical column containing k unique values had k-1 columns created so as to avoid perfect multicollinearity, which is very harmful for logistic regression.

The model itself was sklearn's `LogisticRegression` with the maximum number of iterations set to 5000, which ensures the descent algorithm can converge to find the best coefficients to minimize the cost function.

```
### ------------------- LR PIPELINE -------------------

num_pipeline = Pipeline([
    ('poly', PolynomialFeatures(degree=2, interaction_only=False, include_bias=False)),
    ('scaler', StandardScaler()), # scaling numerical data
])

cat_pipeline = Pipeline([
    ('onehot', OneHotEncoder(drop='first', handle_unknown='ignore')) # onehotencoding ca
])

preprocessor = ColumnTransformer([
    ('num', num_pipeline, num_preds),
    ('cat', cat_pipeline, cat_preds)
])

pipeline = Pipeline([
    ('preprocessor', preprocessor),
    ('model', LogisticRegression(max_iter=5000, n_jobs=-1)) # logistic model
])
```

```
### ------------------- FITTING THE MODEL (TRAIN-TEST SPLIT) -------------------

# Train-test split
X = train_logged.drop(columns=['ON_TIME_AND_COMPLETE'])
y = train_logged['ON_TIME_AND_COMPLETE']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.1, random_state=42

# Fitting the model
model = pipeline.fit(X_train, y_train)
model
```

```
Pipeline(steps=[('preprocessor',
                 ColumnTransformer(transformers=[('num',
```

```
                                                Pipeline(steps=[('poly',
                                                                 PolynomialFeatures(inclu
                                                                ('scaler',
                                                                 StandardScaler())]),
                                                ['PRODUCT_NUMBER',
                                                 'SHIP_FROM_VENDOR',
                                                 'PRODUCT_CLASSIFICATION',
                                                 'AVERAGE_PRODUCT_ORDER_QUANTITY_MARKET'
                                                 'ORDER_QUANTITY_DEVIATION',
                                                 'PURCHASING_LEAD_TIME',
                                                 'PRODUCT_MARKET',
                                                 'AVERAGE_DAILY_D...
                                                                                 handle_unk
                                                ['DIVISION_CODE',
                                                 'COMPANY_VENDOR_NUMBER',
                                                 'ORDER_DAY_OF_WEEK',
                                                 'DUE_DATE_WEEKDAY',
                                                 'GIVEN_TIME_TO_LEAD_TIME_RATIO',
                                                 'DAYS_BETWEEN_ORDER_AND_DUE_DATE_binned
                                                 'AVERAGE_ORDER_CYCLE_DAYS_binned',
                                                 'DISTANCE_IN_MILES_binned',
                                                 'PURCHASE_FROM_VENDOR_binned',
                                                 'TRANSIT_LEAD_TIME_binned'])])),
                        ('model', LogisticRegression(max_iter=5000, n_jobs=-1))])
```

0.3.2 Logistic Model Fine-Tuning

This initial model performed well on the training data - however, it overfitted and performed worse on the test data, losing almost 2 points of accuracy. To overcome this issue, cross-validation and hyperparameter tuning were used.

To begin hyperparameter tuning, `Elasticnet` was implemented for regularization purposes, providing a healthy combination of the L1 and L2 strategies. This helped the model avoid overfitting by shrinking the coefficients and performing feature selection on the interaction and higher-order terms created in the numerical pipeline.

In order to optimize the performance of this elasticnet regression, `GridSearchCV` was employed to find the best combination of hyperparameters. Searching over possible values of the `C` and `l1_ratio` hyperparameters helped improve the model's predictive power.

The cross-validation strategy used for the grid search was a stratified 5-fold cross-validation with shuffling. Although the dataset's response variable is fairly balanced between its two classes, there is a very slight class imbalance, and to be safe, stratified cross-validation was used.

The parameter grid that the model was optimized on included:

- `C` values: [.1, 2, 4, 6, 8]
- `l1_ratio` values: [0.25, 0.5, 0.75]

```
### -------------- GRID SEARCH FOR OPTIMAL HYPERPARAMETERS --------------

# Defining the cross-validation strategy and parameter grid for the grid search
sfk = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
param_grid = {
    'model__penalty': ['elasticnet'],
    'model__solver':['saga'],
    'model__C': [.1, 2, 4, 6, 8],
    'model__l1_ratio': [0.25, 0.5, 0.75]
}

# Conducting the grid search
grid_search = GridSearchCV(estimator=pipeline, param_grid=param_grid, cv=sfk, n_jobs=-1,
grid_search.fit(X_train, y_train)
```

```
GridSearchCV(cv=StratifiedKFold(n_splits=5, random_state=42, shuffle=True),
             estimator=Pipeline(steps=[('preprocessor',
                                        ColumnTransformer(transformers=[('num',
                                                                         Pipeline(steps=[(
                                                                                          (
                                                                                          ['PRODUCT_NUMBER'
                                                                                           'SHIP_FROM_VENDO
                                                                                           'PRODUCT_CLASSIF
                                                                                           'AVERAGE_PRODUCT
                                                                                           'DAYS_BETWEEN_OR
                                                                                           'AVERAGE_ORDER_C
                                                                                           'DISTANCE_IN_MIL
                                                                                           'PURCHASE_FROM_V
                                                                                           'TRANSIT_LEAD_TI
                                       ('model',
                                        LogisticRegression(max_iter=5000,
                                                           n_jobs=-1))]),
             n_jobs=-1,
             param_grid={'model__C': [0.1, 2, 4, 6, 8],
                         'model__l1_ratio': [0.25, 0.5, 0.75],
                         'model__penalty': ['elasticnet'],
                         'model__solver': ['saga']},
             scoring='accuracy')
```

```
# Printing results of the grid search
print("Best hyperparameters:", grid_search.best_params_)
print("Best score:", grid_search.best_score_)
```

```
Best hyperparameters: {'model__C': 8, 'model__l1_ratio': 0.75, 'model__penalty': 'elasticn
Best score: 0.7955468950781184
```

```
# Test accuracy for the grid search
train_accuracy = accuracy_score(y_train, grid_search.predict(X_train))
test_accuracy = accuracy_score(y_test, grid_search.predict(X_test))
print("Train accuracy:", train_accuracy)
print("Test accuracy:", test_accuracy)
```

```
Train accuracy: 0.8092646018180828
Test accuracy: 0.7918707149853085
```

The final, tuned Logistic Regression model resulted in a test accuracy of 79% and a train accuracy of 80%. Although these results are not as high as hoped, they do indicate that the model is ***not overfitting***, since the train and test accuracies are so close. I will try to improve this in the following academic quarter.

0.4 Second Academic Quarter: Logistic Regression, KNN, Random Forest, and XGBoost

In the second academic quarter of this project, I built several other models to become a part of the stacking model, which are outlined below. Many of the models I build in this section already have tuned values for their hyperparameters. These values were found throughout the quarter, through project submissions that are not shown in this report. All of these previously tuned models were built using either Optuna or BayesSearchCV (from Scikit-learn) for hyperparameter tuning. These frameworks were chosen for their flexibility and ability to search over non-discrete grids of hyperparameters, and for their intelligent approaches to learning hyperparameters.

0.4.1 Training and Tuning Base Models

My first base model was a logistic regression, for which I used an elasticnet penalty with tuned values of `C` and `l1_ratio` from the previous quarter's work. For the numeric features, I created polynomial predictors up to degree 2 (both higher-order terms and interaction terms) and completed feature scaling. For the categorical features, I used one-hot-encoding. Although the model is presented in the previous section, I will rebuild the model for the sake of keeping all the base models in one code cell.

The second base model was my KNN submission model from earlier this quarter. I used a K value of 14, along with uniform weights. I found both of these values from my hyperparameter tuning during that submission. My KNN pipeline included similar steps to the logistic regression pipeline.

The third base model I used was a tuned Random Forest from my bagging submission this quarter. The tuned values, visible in the `rf` variable, were found from my hyperparameter tuning in that

model submission. This model's pipeline did not change the numerical features at all, since tree-based models like Random Forests do not benefit at all from monotonic transformations like polynomial features or scaling. Instead, I only one-hot-encoded the categorical features to pass through the Random Forest.

The final, and most powerful, base model that I selected was my tuned XGBoost model from my most recent Boosting submission this quarter. Similarly to the models previously listed, I found the values for hyperparameters in my tuning process during that model's creation. The XGBoost's preprocessing steps were identical to those of the Random Forest, as they are both based on decision trees and do not require any numerical transformations.

```
#### ------------------------ BASE MODELS ------------------------


## ------------ Logistic Regression ------------

# Optimized logistic regression (From my Winter Quarter Prediction Problem)
log_reg = LogisticRegression(
    C=8,
    l1_ratio=0.75,
    penalty='elasticnet',
    solver='saga',
    n_jobs=-1
)

# Creating the logistic regression pipeline
logreg_num_pipeline = Pipeline([
    ('poly', PolynomialFeatures(degree=2, interaction_only=False, include_bias=False)),
    ('scaler', StandardScaler()), # scaling numerical data
])

logreg_cat_pipeline = Pipeline([
    ('onehot', OneHotEncoder(drop='first', handle_unknown='ignore')) # onehotencoding ca
])

logreg_preprocessor = ColumnTransformer([
    ('num', logreg_num_pipeline, num_preds),
    ('cat', logreg_cat_pipeline, cat_preds)
])

logreg_pipeline = Pipeline([
    ('preprocessor', logreg_preprocessor),
    ('model', log_reg)
])


## ------------------ KNN ------------------
```

```
# Optimized KNN (From my KNN submission this quarter)
knn = KNeighborsClassifier(
    n_neighbors=14,
    weights= 'uniform',
    algorithm='auto',
    n_jobs=-1
)

# KNN pipeline
knn_num_pipeline = Pipeline([
    ('poly', PolynomialFeatures(degree=2, interaction_only=False, include_bias=False)),
    ('scaler', StandardScaler()),
])

knn_cat_pipeline = Pipeline([
    ('onehot', OneHotEncoder(drop='first', handle_unknown='ignore')),
])

knn_preprocessor = ColumnTransformer([
    ('num', knn_num_pipeline, num_preds),
    ('cat', knn_cat_pipeline, cat_preds)
])

knn_pipeline = Pipeline([
    ('preprocessor', knn_preprocessor),
    ('model', knn)
])


## ------------- Random Forest -------------

# Optimized Random ForestClassifier (From my bagging submission this quarter)
rf = RandomForestClassifier(
    n_estimators=100,
    max_depth= 15,
    max_features= 0.5,
    max_samples= 0.5,
    random_state=42,
    n_jobs=-1)

# Random Forest pipeline
rf_cat_pipeline = Pipeline([
    ('onehot', OneHotEncoder(drop='first', handle_unknown='ignore')),
])
```

```
rf_preprocessor = ColumnTransformer([
    ('num', 'passthrough', num_preds),
    ('cat', rf_cat_pipeline, cat_preds)
])

rf_pipeline = Pipeline([
    ('preprocessor', rf_preprocessor),
    ('model', rf)
])


## --------------- XGBoost ---------------

# Optimized XGBoost (From my boosting submission this quarter)
xgb = XGBClassifier(
    random_state=1,
    colsample_bylevel= 0.5900125303774969,
    colsample_bynode= 0.5118703775223169,
    colsample_bytree = 1.0,
    gamma = 0.0,
    learning_rate=0.01,
    max_depth = 25,
    min_child_weight=1,
    n_estimators=606,
    reg_alpha=10.0,
    reg_lambda=10.0,
    subsample=1.0,
    n_jobs=-1
)

# XGBoost pipeline
xgb_cat_pipeline = Pipeline([
    ('onehot', OneHotEncoder(drop='first', handle_unknown='ignore')),
])

xgb_preprocessor = ColumnTransformer([
    ('num', 'passthrough', num_preds),
    ('cat', xgb_cat_pipeline, cat_preds)
])

xgb_pipeline = Pipeline([
    ('preprocessor', xgb_preprocessor),
    ('model', xgb)
])
```

After defining the above base models, I decided to evaluate each individually on the training and

test sets that I created previously. The below code cell individually trains each base model and prints their accuracies. As was expected, the best model was the XGBoost, and the worst was the logistic regression. However, stacking fundamentally relies on diverse predictions, so the differences in accuracies between the models, as shown below, do not present a problem for the stacking model. Instead, my stacking model will combine the predictions below in a helpful way.

```
# Creating the list of models
base_models = [
    ('log_reg', logreg_pipeline),
    ('knn', knn_pipeline),
    ('rf', rf_pipeline),
    ('xgb', xgb_pipeline),
]

# Running each model individually, reporting train and test accuracies
results_df = pd.DataFrame(columns=['model', 'train_accuracy', 'test_accuracy'])
for name, model in  base_models:

    # Fitting the model
    model.fit(X_train, y_train)

    # Evaluating and printing the output, also adding to the results dataframe
    train_accuracy = accuracy_score(y_train, model.predict(X_train))
    test_accuracy = accuracy_score(y_test, model.predict(X_test))
    print(f"{name} train accuracy:", train_accuracy)
    print(f"{name} test accuracy:", test_accuracy)
    print("")

    results_df.loc[len(results_df)] = [name, train_accuracy, test_accuracy]
```

```
log_reg train accuracy: 0.7887431277557019
log_reg test accuracy: 0.7683643486777669

knn train accuracy: 0.8007185237602743
knn test accuracy: 0.7722820763956905

rf train accuracy: 0.8778509607533613
rf test accuracy: 0.8139079333986288

xgb train accuracy: 0.8580371237276142
xgb test accuracy: 0.8227228207639569
```

Next, I trained a baseline (no hyperparameter tuning) stacking model. I chose an untuned logistic regression model as the metamodel, and used 5-fold cross-validation as part of the stacking model. The test accuracy achieved by this untuned model was 0.821743388834476.

```
# Using a logistic regression metamodel to aggregate each individual base learner
metamodel = LogisticRegression()

# defining a cross-validation strategy
cv = KFold(n_splits=5, shuffle=True, random_state=1)

# Fitting the stacking model
stack = StackingClassifier(
    estimators=base_models, # list of base models as the stacking estimators
    final_estimator=metamodel, # logistic regression meta model
    stack_method='auto', # passing probabilities (not classes) to the meta model
    cv=cv, # 5 fold cross validation with
    n_jobs=-1, # parallelize jobs
    passthrough=False, # do not pass original data to meta model (only base model predic
)
stack.fit(X_train, y_train)

train_accuracy_stack = accuracy_score(y_train, stack.predict(X_train))
test_accuracy_stack = accuracy_score(y_test, stack.predict(X_test))
print("Stacking train accuracy:", train_accuracy_stack)
print("Stacking test accuracy:", test_accuracy_stack)
```

```
Stacking train accuracy: 0.8668553698764356
Stacking test accuracy: 0.821743388834476
```

0.4.2 Stacking Model Hyperparameter Tuning

For the hyperparameter tuning step of the final model, I chose to tune the metamodel of my stacking model. I decided to use Optuna for its quick, smart, and easy-to-implement tuning process. I tuned the values of `C` (inverse regularization coefficient) and the `l1_ratio` in order to regularize my stacking model's aggregation of its base models. This is implemented to help the model avoid any multicollinearity issues that arise due to similarities in the base model's predictions. I iteratively selected the tuning space and ran the below code cell many times. After each trial, I examined the three plots that are printed below the model's output. Specifically, I used the third graph - which plots each hyperparameter against the objective function - to iteratively narrow down my search space to find regions that Optuna could find higher accuracies on.

After a long process of trial and error, I achieved a cross-validation accuracy of 0.8229817618936524. The hyperparameters used on the metamodel to achieve these accuracies are:

- `C`: 0.20625358439143332
- `l1_ratio`: 0.7465115373496148

```
warnings.filterwarnings("ignore")
```

```
# Optuna objective function
def objective(trial):

    # Tuning the metamodel C and l1_ratio
    C = trial.suggest_float('C', 1e-4, 10)
    l1_ratio = trial.suggest_float('l1_ratio', 0.1, 1.0)

    # Setting the final estimator on the stacking model
    final_estimator = LogisticRegression(
        penalty='elasticnet',
        solver='saga',
        C=C,
        l1_ratio=l1_ratio,
        max_iter=500
    )

    # Implementing the stacking classifier with the final_estimator that uses the sugges
    stack = StackingClassifier(
        estimators=base_models, # list of base models as the stacking estimators
        final_estimator=final_estimator, # logistic regression meta model
        stack_method='auto', # passing probabilities (not classes) to the meta model
        cv=cv, # 5 fold cross validation with
        n_jobs=-1, # parallelize jobs
        passthrough=False, # do not pass original data to meta model (only base model pr
    )

    # Returning mean cross-validated accuracy score
    scores = cross_val_score(stack, X_train, y_train, cv=cv, scoring='accuracy')
    return scores.mean()

# Running the optuna study
study = optuna.create_study(direction='maximize')
study.optimize(objective, n_trials=20, )
```

```
# Outputting the results of the study
best_params_stack = study.best_params
print("Best metamodel params:", best_params_stack)
print("Best CV accuracy:", study.best_value)
```

```
Best metamodel params: {'C': 0.20625358439143332, 'l1_ratio': 0.7465115373496148}
Best CV accuracy: 0.8229817618936524
```

```
# Plotting the optimization history to see which trials were the most successful
vis.plot_optimization_history(study)
```

Unable to display output for mime type(s): application/vnd.plotly.v1+json

```
# Plotting the hyperparameter importances to see which ones mattered the most
vis.plot_param_importances(study)
```

Unable to display output for mime type(s): application/vnd.plotly.v1+json

```
# Plotting the objective values against each of the hyperparameter spaces
vis.plot_slice(study)
```

Unable to display output for mime type(s): application/vnd.plotly.v1+json

0.5 Final Model Training And Prediction

```
# Best Metamodel, with the parameters found from the Optuna study.
best_metamodel = LogisticRegression(
    penalty='elasticnet',
    solver='saga',
    C=best_params_stack['C'],
    l1_ratio=best_params_stack['l1_ratio'],
    max_iter=500
    )

# Creating the best stacking model
stack_final = StackingClassifier(
    estimators=base_models, # list of base models as the stacking estimators
    final_estimator=best_metamodel, # logistic regression meta model
    stack_method='auto', # passing probabilities (not classes) to the meta model
    cv=cv, # 5 fold cross validation with
    n_jobs=-1, # parallelize jobs
    passthrough=False, # do not pass original data to meta model (only base model predic
)
stack_final.fit(X_train, y_train)

# Ouputting the results of the final model
train_accuracy = accuracy_score(y_train, stack_final.predict(X_train))
test_accuracy = accuracy_score(y_test, stack_final.predict(X_test))
print("Tuned Stacking Model Results:")
print("Train accuracy:", train_accuracy)
print("Test accuracy:", test_accuracy)
```

```
Tuned Stacking Model Results:
Train accuracy: 0.8662021664580045
Test accuracy: 0.8227228207639569
```

The final model has a test accuracy of 82.27%, which is an improvement on the single-tuned logistic regression model.

0.6 Conclusion and Key Takeaways

My final model was slightly better than my previously submitted models. The two models had very comparable cross-validation accuracy (both around 0.82). The improvement from my boosting model to this new stacking model is very small, likely attributable to the fact that there were only four base learners that were a part of my stacking model. Adding a higher number of more diverse base learners - such as different tree-based models, support vector machines, and even neural networks - would likely result in an even larger increase in accuracy from my boosting model. Stacking benefits the most when its base learners have extremely diverse and uncorrelated predictions. Adding these other base learners would likely increase the stacking model's diversity and improve its performance.

One notable takeaway is that the stacking model did not improve its test accuracy upon the strongest of the base learners, the xgboost model. In this case, **I would advocate for the use of the sole xgboost model**, since it is much more computationally efficient than the stacking model without sacrificing any accuracy.